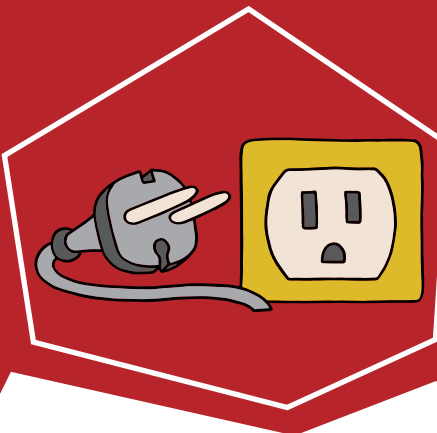


problem time

Keep Solving Problems and Deliver Value

MANAGEMENT 3.0 © 2016 JURGEN APPELO



WWW.MANAGEMENT30.COM/PROBLEM-TIME

It's so much easier to suggest solutions when you don't know too much about the problem.

Malcolm Forbes,
American publisher
(1919–1990)

It is common for organizations to focus on delivery times, but instead, they should be focusing on their queues and work-in-progress. By measuring total problem time and average problem time on improvement boards, workers can train themselves to keep solving problems, improve their work, and make all clients and stakeholders happy.

Instead of focusing on the happiness of people,
we should be focusing on the improvement of work.

After we landed in Buenos Aires, we went through immigration, picked up our luggage from baggage claim, flung everything through the giant security machines, and went straight to the two cash machines at the first bank we saw in the arrivals hall to withdraw a pile of pesos. Unfortunately, the first machine we tried didn't work, and the second one was broken. There were three machines from another bank around the corner; the first was out of order, the second had a personality problem, and the third thought it was a statue in loving memory of times gone by when people were able to get cash.

It turned out that *all* the cash machines at the airport in Argentina refused to give us any money. One would think that a high priority in any country would be to ensure that tourists were able to transport foreign money into the country. But not in Argentina. The rich diversity of error messages and dysfunctional behaviors of cash machines Raoul and I encountered during our ten-day trip could turn any software maintenance person into a technical paleontologist. It turned us into gamblers because the complete randomness of pay-outs made the cash machines seem more like slot machines.

We encountered a few more problems on our trip in Argentina. The first *oficina de cambio* (exchange office) we visited in the city center didn't have any pesos. Figuring out where to buy a bus ticket in Bariloche took fifteen minutes of asking around at various kiosks within a hundred meter radius of the bus stop. Returning a rental car usually takes us five minutes, but in Bariloche, it took half an hour. Getting into an airport lounge anywhere in the world requires little more than offering a boarding pass to the receptionist; in Buenos Aires it took five minutes of browsing through a stack of papers, checking numbers, and filling out passenger data, by hand. And the priority check-in and boarding queues for frequent flyers probably exist somewhere,

but you'll have to ask around. (Or just create your own, like we did.) None of this seems to bother the Argentinians that much, because research shows they are quite happy. [Happy Melly, "The Future of Happiness Can Be Found in South America"; LiveScience, "The Happiest Countries"] And they love dancing.

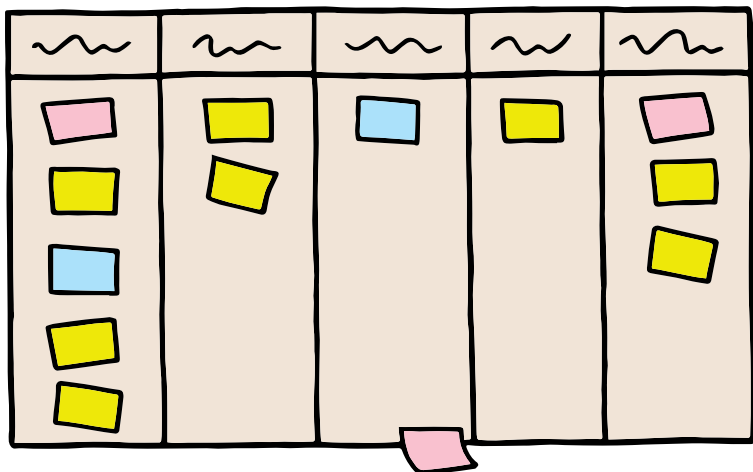
My experiences confirm the hypothesis that a person's happiness doesn't necessarily lead to an improvement in a person's work. Some writers claim that organizations should focus more on the happiness and well-being of their workers, because happiness leads to higher productivity and better performance. [Heathfield, "Are Your Employees Happy at Work?"] There is probably some truth in there, but there is also evidence that a much *stronger* correlation exists the other way around: When organizations perform better, the employees' sense of happiness and well-being increases! [Rosenzweig, *The Halo Effect* loc:1349]

I strongly believe improvement of work leads to more happiness for everyone. Raoul and I very much enjoyed the sun, chorizo steaks, ice cream, the tango, wine, and the beautiful scenery in and around Buenos Aires and San Carlos de Bariloche. In terms of enjoyment we might rate our vacation as 9 out of 10. It *could* have been 10 out of 10 if the coffee house we found in San Martin de los Andes during an eight-hour road trip on our last day was accompanied by a coffee machine that actually worked.

These observations have convinced me that, instead of focusing on the *happiness of people*, possibly supported with a **happiness index** [Sutherland, "Happiness Metric"], we should be focusing on the *improvement of work*, preferably with some kind of improvement index. When things improve, people will be happier.

Solving Problems

Argentina is not the only place in the world with user problems. For example, whenever I try to use Twitter's website for more than a few minutes, I often get a strong desire to go and live in Argentina. At least, over there, I could have a nice juicy steak while dealing with things that are not working. I'm convinced that many other organizations *and* countries (including mine) can use a bit of improvement as well.



There has been a trend in the last ten years to visualize continuous improvement with **improvement boards** (or problem walls or impediment backlogs). Experts suggest that employees keep track of the problems (or the-things-that-can-be-improved) in an organization by visualizing them with sticky notes placed somewhere in the workplace. [Beaver, "Build a Team Improvement Backlog"] These boards can contain issues such as "update employee contracts",

"enable Wi-Fi in client waiting room", "pay invoices faster", "move all websites to a more stable hosting provider", or "repair coffee machine".

It is important to emphasize that improvement boards should not cover regular product requirements or service requests. Those types of issues can be addressed with their own specific task boards and can be managed with Scrum [Rubin, *Essential Scrum*], Kanban [Anderson, *Kanban*], or some other work flow management framework. The purpose of improvement boards is to keep track of systemic problems reported by clients and stakeholders regarding the work of a whole team, department, business unit, or organization. Adding a new button to a website is a functional requirement. Not seeing the new button because the website is always off-line is a problem begging for improvement. One cash machine running out of money indicates the need for a regular service request. Having five defective cash machines at the same time, with nobody around to fix them, indicates a systemic problem in the organization.

The concept of visualizing problems on a wall, so that everyone can see them, is a great idea. We often call this type of visualization **information radiators**, or Big Visible Charts. [Cockburn, *Agile Software Development*] However, I have noticed two important issues with this practice.

The first issue is that many people have the talent to completely ignore the most obvious information that has been radiating for a long time. I once attached a big, colorful note to the inside of the front door of my house as a reminder not to forget my wallet, my keys, my phone, and several other items I often left at home. The note worked—for one week. But after a week, I didn't see the note

Many people have the talent
to completely ignore the most
obvious information.

anymore. It had become part of my normal environment. It stopped being remarkable. The same applies to the tiny pair of wooden shoes I have dangling from the front window of my car. I put them there as a reminder to keep track of the car's mileage. It worked for a few days, but then the shoes became just another car accessory. Likewise, there are plenty of improvement boards in the world radiating harder than the Argentinean sun, but, after a promising start, they often stopped energizing anyone to improve anything.

The second issue with information radiators is that more and more people are working remotely in the globalized economy of the 21st century. Many creative networkers don't share the same office with their co-workers. As a result, they cannot pick up the same visual clues. They must rely more and more on digital information stored in online tools. Granted, this is far from ideal, and if you can use physical information radiators with a collocated team you definitely should try. But to improve all organizations, we cannot get away with the simplistic suggestion that all teams should be collocated in the same work space and that all teams should have big visible charts on their walls.

Considering that we cannot simply rely on office workers to properly pick up the information about continuous improvement radiated by their improvement boards, and that some of us may not even be in the office, I believe we need something else to guide us toward improvement of our work.



Unsolved Problems

If we want problems solved, we have to do a bit more than just visualize them with sticky notes. We have to perform some actual **problem management**. And to *manage* problems, a good start would be to *measure* them. 📌 Peter Drucker once famously said, “What gets measured gets managed.” Therefore, a good question is, “How can we measure our problems in an effective way so that they get solved and things will improve?” [Seddon, *Freedom from Command & Control* pag:44]

Measurement is fundamental to high performance, improvement, and, ultimately, success in business, or in any other area of human endeavor.

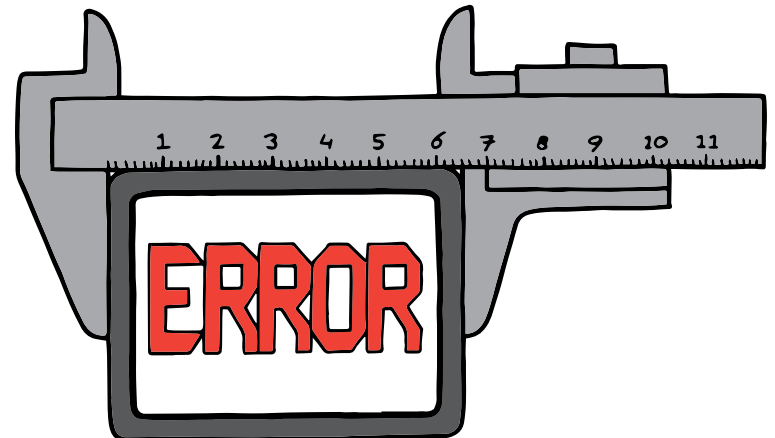
Spitzer, *Transforming Performance Measurement* loc:199

A common mistake is to assume that the *number of problems* is a decent key performance indicator of organizational dysfunction. After all, the thinking goes, the more problems we have on the backlog the worse our performance must be. However, this is not necessarily true. By measuring and reporting the number of problems in a distrustful environment, we easily get into a situation where people feel pressured not to add more problems to a large queue in order to prevent the metric from growing even larger. The result is a visible backlog of *reported* and *managed* problems, and an invisible backlog of *unreported* and *unmanaged* problems. (This might be the situation at some Argentinian banks, but I’m just guessing here.)

Another mistake is to think that when the queue size is stable, we have things under control. Before anything else, we must keep in mind the

perspective of the stakeholder. What does anyone want who reports a problem? They want their problem to be fixed, and sooner rather than later! When *this* week’s problem queue is exactly the same as *last* week’s problem queue, does that mean our performance has remained the same as the week before? No! The people who reported the problems have now been waiting an extra week for us to fix them! Therefore, the metric we use should reflect that our performance has worsened. Any metric we come up with should incorporate the *age of problems*.

We should not penalize anyone for reporting problems. But we *should* penalize ourselves for not solving those problems rapidly. Finding one defective cash machine at an airport is a minor inconvenience. The machine could have broken down an hour ago. But finding *five* broken machines indicates a severe organizational dysfunction. And if those same machines are all still broken the following week, things are even worse than we thought! The longer a problem remains unsolved, the heavier this must weigh on our improvement index.



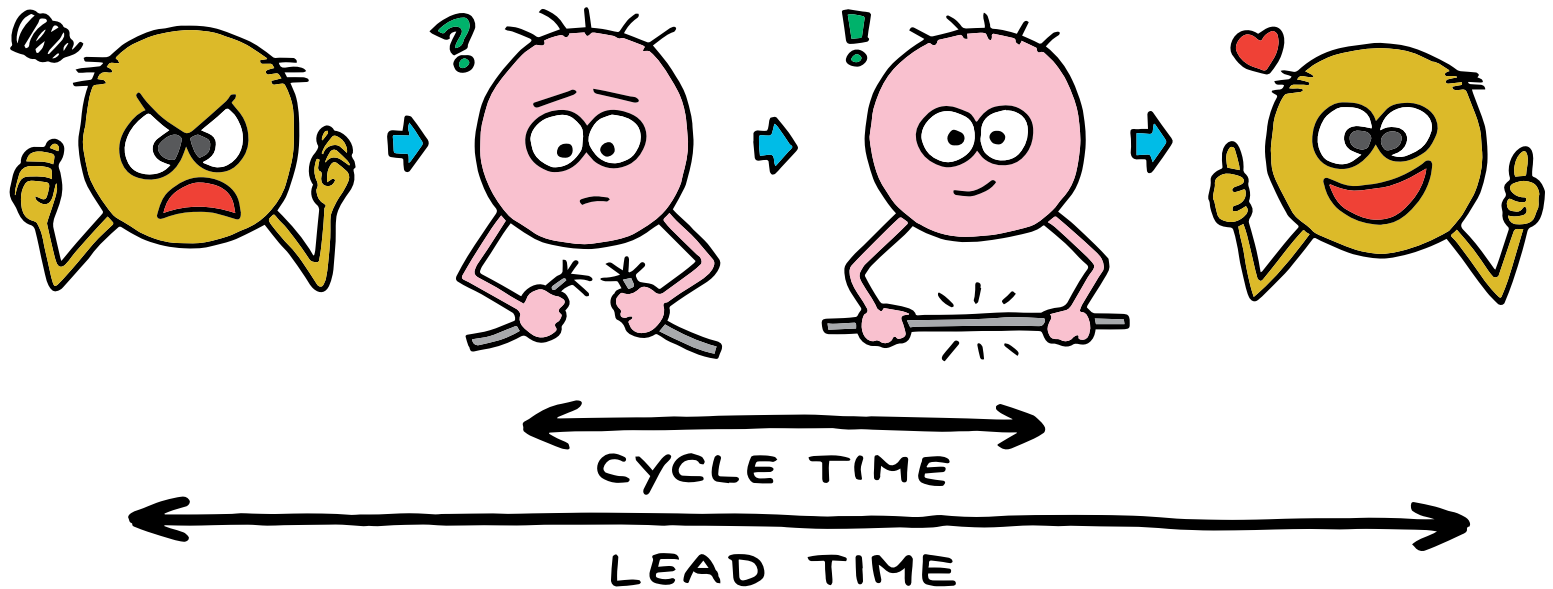


Work-In-Progress

Sadly, most organizations don't pay attention to queues. Instead, they pay attention to the time it takes to give customers what they want, or the time it takes to develop something. In a way, this makes sense because customers don't care about the size of queues. They care about their time.

Most organizations don't pay attention to queues.

Two metrics are often mentioned in performance management literature: **lead time** and **cycle time**. Lead time is the time measured from the moment a customer *reports* an issue until the moment the customer considers the matter solved. Cycle time is the time measured from the moment *work starts* to address an issue until the moment the organization considers the matter closed.  Logically, cycle time is always shorter than (or equal to) lead time. [Roock, "Lead Time and Cycle Time"]



Like queue size, lead time and cycle time are useful metrics, and like queue size, the metrics suffer from a few issues. One important problem with lead time and cycle time is that the data only becomes available once the problems have been solved. These metrics are a typical example of **lagging indicators**. They are only known *after* you're done. Consider the example of long lines in a supermarket. By only measuring lead time and cycle time, you will not know about the build-up of your customers' frustration *inside* the supermarket. You will only know about the long time they have been waiting in the queues *after* they have slammed the doors behind them on their way out—possibly never to return.

Another issue with lead time and cycle time is that these measurements originate from the manufacturing sector and are used to manage inventory of unsold physical goods, such as cars and books. Unsolved problems in an organization can be treated metaphorically *as if* they are unsold inventory, but reported problems are definitely not the same thing as physical inventory. The inventory metaphor breaks down easily. For example, two reported problems could later turn out to be the same problem viewed from different angles. I've never heard of anyone merging two unsold cars!

The third and most important argument against a focus on lead time and cycle time comes from queuing theory. It appears that measurement of queues is much more effective than measurement of waiting time. Again, consider supermarkets: all successful supermarkets have figured out that they must monitor the lines of people waiting to pay and keep those lines small. Customers regret the loss of the *time* they have spent waiting, and if the organization keeps its focus on *queues*, it turns out that people's waiting times will drop automatically. And information about queue size is available long *before* lead time and cycle time. It is a **leading indicator** of the happiness of clients.

Queues are a far better control variable than cycle time because [...] queues are leading indicators of future cycle time problems. By controlling queue size, we automatically achieve control over timelines.

Reinertsen,
The Principles of Product Development Flow loc:296

Unfortunately, most organizations do not monitor the size of queues or the number of problems that have been reported to them. If they measure anything at all, it is usually the time it took them to solve a specific customer's problem or the time it took to develop something. For most workers, it requires a complete mindset shift to change focus from lead time and cycle time to their **work-in-progress (WIP)**. When we measure and manage our work-in-progress, the waiting times for clients will take care of themselves. [Reinertsen, *The Principles of Product Development Flow* loc:2264]

It requires a complete mindset shift
to focus on work-in-progress (WIP).

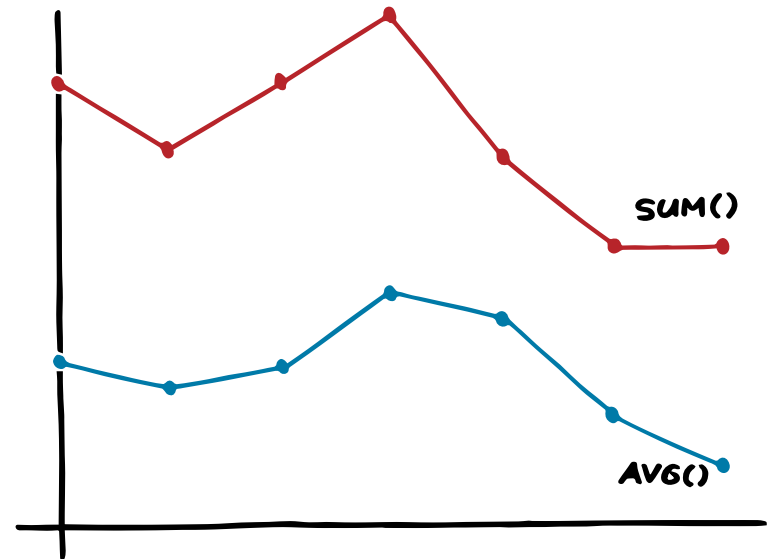
Performance Measurement

We've seen that measuring queue size (or WIP) is better than measuring lead time and cycle time, but a static queue size does not express the growing frustration among stakeholders regarding the aging of their unsolved problems. And visualizing queue size alone does not incentivize people to report new problems.

As someone who is *responsible* for the system, I value three things:

1. *Reported problems are better than unreported problems.*
I want all problems to be reported. No problem should be kept hidden. People should feel safe and incentivized to report any issue they find.
2. *Young problems are better than old problems.*
I want problems to be solved fast. They should not linger around on backlogs and boards for long because frustration grows with the age of problems.
3. *Non-recurring problems are better than recurring problems.*
I don't want the same problems to pop up again and again. Once solved, they should remain solved for good. Permanent fixes are preferable over short-term workarounds.

Given these three requirements, I believe we should measure *the total age of all problems*. Every week, we can spend a few minutes evaluating the entire list of open problems which have been posted on a wall or stored in a shared online tool. We then add a dot (or a point or a plus) to each problem that is still open. The number of dots (or points or pluses) per problem indicates how long this issue has been waiting to be solved. We can only remove a problem when the person who reported it agrees that the matter has been resolved. And, most

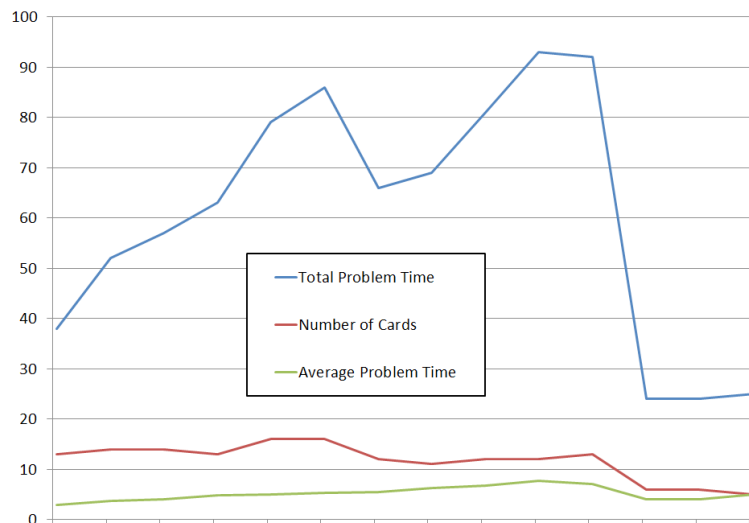


importantly, when the same problem is reported again by another client, we re-introduce the issue in the queue, *starting with its former number of dots*. (Apparently, the problem was not properly solved!)

Once per week—or more often, depending on the nature of your business—we calculate the *sum* and *average* of all the dots  and we report the results to everyone who is part of the system. What you measure is what you get, and with these two metrics, **total problem time** and **average problem time**, we get exactly what we want: people feel an incentive to *report* new problems, partly because adding fresh problems can bring the average problem time down (but not the total sum). They also feel an incentive to *solve* problems on the board because this brings the total problem time down (but not necessarily

the average). And when creative networkers focus mainly on solving problems that have already been on the backlog for a long time, they will reduce both the total time *and* the average problem time. Last, but not least, there is an incentive to solve problems for good because this prevents them from reappearing on the backlog. And we don't want our clients to keep encountering the same problems, do we?

Problem time is different from queue size because problem time can increase while queue size remains static, indicating a (possibly) growing frustration of clients who are waiting for their problems to be solved. Problem time is also different from lead time and cycle time because lead time and cycle time are measures of *completed* work—they are lagging indicators—while problem time has an exclusive focus on *uncompleted* work—a leading indicator.



What about a limited Work-in-Progress?

Many experts suggest that workers should keep the amount of work small by imposing a limit on their queues. This is often called a **limited wip**. [Anderson, *Kanban* loc:2497] It certainly makes sense in many situations to keep a clear focus and to refrain from unnecessary task-switching, which is bad for productivity.

However, though a well-defined limit can be healthy for the stuff you're *working on*, it doesn't sound like a good idea for the number of *problems reported*. A hospital may have a policy for the maximum number of people being treated in Intensive Care, but when you arrive at their doors with a severed finger, it is unlikely they will deny you access with the reply, "Sorry, we've reached our limited wip for today. Please come back tomorrow." That would qualify as bad service, not as a smart policy.

Improvement Technique

Let's return to our stakeholders for a moment. They are usually outsiders *interacting* with our organization. They can be customers, suppliers, shareholders, representatives of local communities, or everyone else who has a problem with our organization which can't be solved without our help. Our stakeholders also have three requirements, which are, unsurprisingly, very similar to ours:

1. *An easy-to-report problem is better than a hard-to-report problem.*
When they encounter a problem, they want to report it in the easiest possible way, for example, without filling out a complicated form.
2. *A solved problem is better than an unsolved problem.*
After reporting a problem, it should be fixed as soon as possible, preferably accompanied by an apology, thank-you-very-much.
3. *A unique problem is better than a well-known problem.*
Our clients actually prefer that problems don't exist at all, and they certainly don't want them to reappear again.

Of these three requirements, the only one we still need to take care of is the first. The second requirement (they don't want old unsolved problems) and the third requirement (they don't want recurring problems) are already covered indirectly by our two problem metrics: total problem time and average problem time. But I think the first requirement (they want to give feedback easily) needs special attention. We can address it by always asking our clients the golden question. 



Is there anything we can do better?

The barrier for people to give feedback must be as low as possible. If we don't get an answer to our golden question we should consider that itself a problem! On a number of occasions I have left a restaurant or hotel in an angry mood, vowing never to return again. When nobody asks me if everything was fine—in Europe nobody usually does—the business will never know they lost a customer forever. And I'm not the kind of person who wastes ten minutes of his valuable time on a company's evaluation form when its product or service has already annoyed me tremendously. Sending out a rant on Twitter is much easier and more satisfying.

There is not much more we need to do in order to manage improvement and increase the level of happiness for everyone. All we have to do is ask our stakeholders, "Can we do better?" and add any newly reported issues to our problem board. By tracking the total time and average time of the reported problems on the board, we train ourselves to solve them quickly and solve them forever.



Problem Definition

Now that we know how to measure and manage problems, there are just two more questions for us to discuss. The first one is:

What exactly is a problem?

Who decides whether something is a problem or not? The client does! When I'm in a hotel room and I perceive the lack of an electrical socket next to my bed as a problem, then there *is* a problem. When I find only one pillow on my bed, and it's as flat as the Dutch countryside on a rare sunny day, then there *is* a problem. When I notice that the "high-speed" hotel Wi-Fi is more congested than a French highway on Black Saturday, then there *is* a problem!

The customer is not in a good position to prescribe product or service that will help him in the future.

Deming, Out of the Crisis pag:167

Notice that a client perceiving a problem is *not* the same as "the customer is always right". Maybe electrical sockets in walls are impossible in an old building that is a protected monument. (I would be perfectly happy with a long extension cord.) Maybe paper-thin pillows are an expression of local culture. (Having a bigger one available in the wardrobe would be an easy solution.) Maybe high-speed Wi-Fi is impossible deep in a forest or high on a mountain top. (A note about this while booking the room would be appreciated.) The problem, as perceived by a customer might not be a problem that can be fixed. But when a customer feels annoyed by something, then this *is* still a problem!

The same applies to suppliers, shareholders, employees, and all other clients. When some of them are unhappy, we have a problem! And we need to figure out a way to make them feel good again about our business.

Also, notice that the things workers are trying to do are often *not* described by them in terms of the needs of their stakeholders. I'm quite sure that no client has asked you to "identify metrics for continuous improvement", to "develop a training plan for new employees", or to "upgrade the website platform to version 4.2". And I hope no client has ever asked you to "update the competence matrix in the performance appraisal template". Yes, some of these tasks might be necessary to solve someone's problem. But the thing that should be on your improvement board before anything else is the client's *problem*, not just your *tasks*. Yes, tasks are useful. Tasks are good. Tasks make things happen. But you have to remind yourself why you're doing them in the first place. What should be ticked off as "solved" on your improvement board is the stakeholder's problem.

That brings me to the last topic to be addressed, which is:

When is a problem solved?

There's only one good answer to that, which comes from the mouth of the client standing in front of a nonworking cash machine: "It is solved when I say so!" Yes, it might be painful to hear this, but we *need* clients like that. The world has never been changed by those who just accept bad service or bad products.

The world has never been changed
by those who just accept bad
service or bad products.

A problem is only really solved when the client feels good about you and your business. (In some cases, this could even mean two parties happily going their separate ways.) This indicates, again, that continuous improvement should ultimately lead to happiness. It can be fun to measure happiness, but you won't make things better by scrutinizing your happiness index. You make things better by improving your

It is not sufficient to improve processes. There must also be constant improvement of products, services and technologies.

Deming, Out of the Crisis pag:135

processes, products, services, and everything else. Therefore, what you should monitor closely is the collective and average age of all problems, and whether they are found and reported easily. By doing this, you not only engage the people, you also improve the work and delight all clients. And that is all that we want.



How to Get Started

It shouldn't be that hard to get started measuring problem time in your team. You can start with your existing improvement backlog, if you have one, or you can create a new one from scratch.

1. Make sure your improvement board contains stakeholder problems, not just tasks.
2. Check or estimate when each problem was added to the board, and calculate a problem time per issue.
3. Calculate the total and average problem time for the entire board, and communicate it to your team members. Make a recurring task in your task list to do this every week from now on.
4. Start asking stakeholders "What can we do better?" on a regular basis.



References

- Anderson, David J. Kanban: Successful Evolutionary Change in Your Software Business. Sequim: Blue Hole Press, 2010. Print.
- Beaver, Jan. "Build a Team Improvement Backlog" <<http://bit.ly/1dpGxPy>> CoachingAgile, 10 December 2010. Web.
- Cockburn, Alistair. Agile Software Development: The Cooperative Game (2nd Edition). Upper Saddle River: Addison-Wesley, 2007. Print.
- Deming, W. Edwards. Out of the Crisis. Cambridge: Massachusetts Institute of Technology, Center for Advanced Engineering Study, 1986. Print.
- Happy Melly. "The Future of Happiness Can Be Found in South America" <<http://bit.ly/1htkRWj>> Happy Melly, 3 February 2014. Web.
- Heathfield, Susan M. "Are Your Employees Happy at Work?" <<http://abt.cm/1b0cviQ>> About.com, 21 July 2013. Web.
- LiveScience. "The Happiest (and Saddest) Countries" <<http://bit.ly/1j75Nwj>> LiveScience, 9 September 2013. Web.
- Reinertsen, Donald G. The Principles of Product Development Flow: Second Generation Lean Product Development. Redondo Beach: Celeritas, 2009. Print.
- Roock, Stefan. "Kanban: Definition of Lead Time and Cycle Time" <<http://bit.ly/K9p1VN>> Stefan Roock, 2 March 2010. Web.
- Rosenzweig, Philip M. The Halo Effect and the Eight Other Business Delusions that Deceive Managers. New York: Free Press, 2007. Print.
- Rubin, Kenneth S. Essential Scrum: A Practical Guide to the Most Popular Agile Process. Upper Saddle River: Addison-Wesley, 2012. Print.
- Seddon, John. Freedom from Command & Control: Rethinking Management for Lean Service. New York: Productivity Press, 2005. Print.
- Spitzer, Dean R. Transforming Performance Measurement: Rethinking the Way We Measure and Drive Organizational Success. New York: American Management Association, 2007. Print.
- Sutherland, Jeff. "Happiness Metric – The Wave of the Future" <<http://bit.ly/1gExE6u>> Scrum Inc., 22 June 2012. Web.



Jurgen Appelo

Managing for Happiness

Games, Tools & Practices to Motivate Any Team



Available
from
June
2016

“Jurgen's book is practical and fun, but most of all, it's subversive. If you care enough to get started, you'll discover that these tools will transform everything about your organization.”

Seth Godin, *The Icarus Deception*

